

 Se former autrement	Devoir Surveillé Semestre : 1 ☒ 2 ☐ Session : Principale ☒ Rattrapage ☐
Module : Théorie des Langages Enseignant(s) : Equipe TLA Classe(s) : 3A29-3A63 Documents autorisés : OUI ☐ NON ☒ Nombre de pages : 2 Calculatrice autorisée : OUI ☐ NON ☒ Internet autorisée : OUI ☐ NON ☒ Date : 31/10/2025 Heure : 9h Durée : 1h	

Exercice 1. Analyse lexicale pour requêtes SQL basiques (6pts)

On souhaite définir un analyseur lexical à l'aide de Flex pour un langage de requête SQL simple qui reconnaît les éléments suivants :

- **Commandes SQL** : SELECT, FROM, WHERE
- **Nombres entiers**: composés uniquement de chiffres
- **Noms de colonnes/tables** : composées de lettres
- **Opérateurs logiques** : =, >, <
- **Mots-clés logiques** : AND, OR
- **Symboles** : , ;

Exemples de requêtes:

```
SELECT nom, age FROM clients WHERE age > 18;
```

```
SELECT produit FROM stock WHERE quantite < 50 OR prix = 100;
```

1. Compléter les parties manquantes du fichier de spécification Flex suivant pour construire un analyseur lexical reconnaissant les éléments mentionnés ci-dessus et permettant de retourner sur la console, à chaque identification d'un lexème, la chaîne reconnue ainsi que la description. (2pts)

```
%{
    #include <stdio.h>
    #include <stdlib.h>
}%
.....
.....
%%
..... printf(.....);
..... printf(.....);
..... printf(.....);
..... printf(.....);
..... printf(.....);
..... printf(.....);

%%
int main(){
    yylex();
    return 0 ; }
```

2. Pourquoi doit-on définir la règle des **commandes SQL** avant celle des **noms de colonnes/tables**? (1pt)
3. Que se passe-t-il lorsqu'aucune règle ne permet de reconnaître un symbole rencontré dans le texte d'entrée? Proposez une solution pour gérer ce type de situation. (1pt)
4. Pourquoi l'analyseur lexical ne vérifie-t-il pas que SELECT est toujours suivi de FROM ? (1pt)
5. Analyser la sortie produite pour la requête suivante: `SELECT @nom FROM #clients;` (1pt)

Exercice 2. Expression Régulière et Automate (6pts)

1. Pour chaque paire, déterminer si les expressions régulières sont équivalentes et justifier brièvement: (2pts)

- 1.1) $(a^*b^*)^*$ et $(a|b)^*$
- 1.2) $a(ba)^*b$ et $(ab)^+$
- 1.3) $a^*|b^*$ et $(a|b)^*$
- 1.4) $(aa)^*a$ et $a(aa)^*$

2. Soit $\Sigma = \{a, b\}$ et le langage $L = \{w \in \Sigma^* \mid w \text{ commence par 'a' et } |w|=2k, k>0\}$

- 2.1) Donner l'expression régulière pour L (1pt)
- 2.2) Construire un automate fini non-déterministe sans ϵ - **transition** pour L (2pts)
- 2.3) Déterminer l'AFN obtenu à la question 2.2 (1pt)

Exercice 3. Thompson (8pts)

Soit le langage **L** défini par l'expression régulière suivante : $(a|b)^*ab(a|b)^*$

1. Utiliser l'algorithme de Thompson pour construire un automate fini non déterministe (AFN) reconnaissant le langage **L** (2pts)
2. Déterminer l'automate que vous avez construit (2.5pts)
3. Donner l'automate minimal correspondant (2pts)
4. Dédire l'automate du langage **L'** des mots qui ne contiennent pas le facteur "ab". (1.5pt)